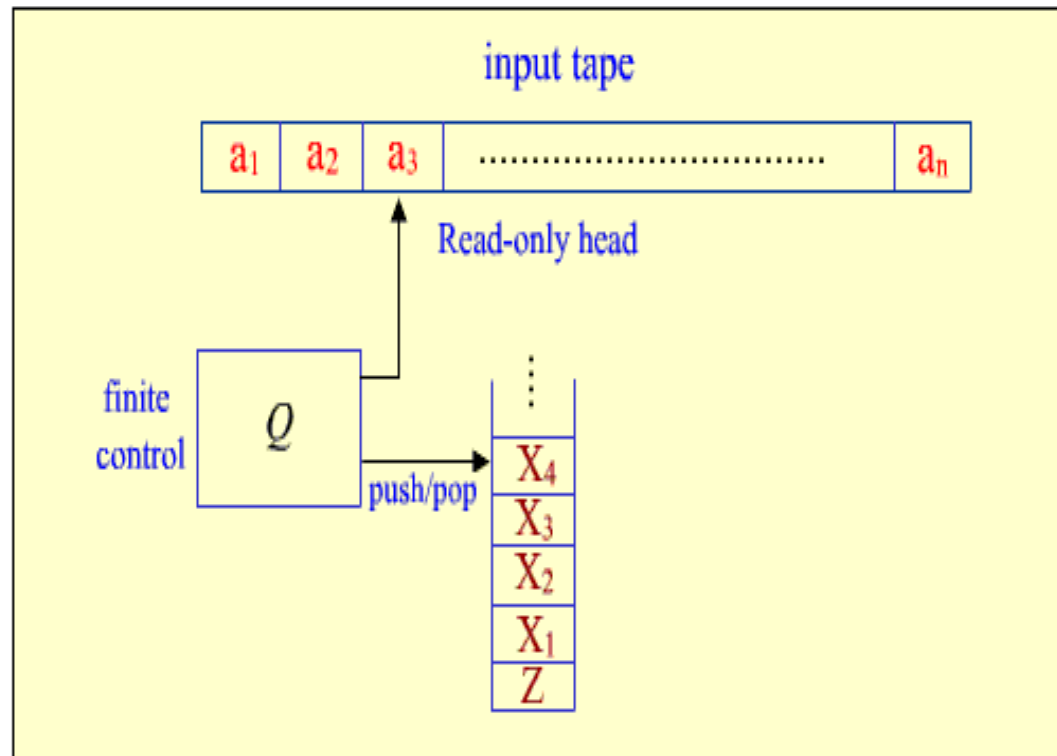


UNIT-III
PUSHDOWN AUTOMATA
&
TURING MACHINE

Regular language can be characterized as the language accepted by finite automata. Similarly, we can characterize the context-free language as the language accepted by a class of machines called "[Pushdown Automata](#)" (PDA). A pushdown automaton is an extension of the NFA.

It is observed that FA have limited capability. (in the sense that the class of languages accepted or characterized by them is small). This is due to the "[finite memory](#)" (number of states) and "[no external memory](#)" involved with them. A PDA is simply an NFA augmented with an "[external stack memory](#)". The addition of a stack provides the PDA with a [last-in, first-out](#) memory management capability. This "[Stack](#)" or "[pushdown store](#)" can be used to record a potentially unbounded information. It is due to this memory management capability with the help of the stack that a PDA can overcome the memory limitations that prevents a FA to accept many interesting languages like $\{a^n b^n \mid n \geq 0\}$.

Although, a PDA can store an unbounded amount of information on the stack, its access to the information on the stack is limited. It can push an element onto the top of the stack and pop off an element from the top of the stack. To read down into the stack the top elements must be popped off and are lost. Due to this limited access to the information on the stack, a PDA still has some limitations and cannot accept some other interesting languages.



As shown in figure, a PDA has three components: an input tape with read only head, a finite control and a pushdown store.

The input head is read-only and may only move from left to right, one symbol (or cell) at a time. In each step, the PDA pops the top symbol off the stack; based on this symbol, the input symbol it is currently reading, and its present state, it can push a sequence of symbols onto the stack, move its read-only head one cell (or symbol) to the right, and enter a new state, as defined by the transition rules of the PDA.

PDA are nondeterministic, by default. That is, ϵ -transitions are also allowed in which the PDA can pop and push, and change state without reading the next input symbol or moving its read-only head. Besides this, there may be multiple options for possible next moves.

Formal Definitions : Formally, a PDA M is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

where,

- Q is a finite set of states,
- Σ is a finite set of input symbols (input alphabets),
- Γ is a finite set of stack symbols (stack alphabets),
- δ is a transition function from $Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma$ to subset of $Q \times \Gamma^*$

- $q_0 \in Q$ is the start state
- $z_0 \in \Gamma$, is the initial stack symbol, and
- $F^* \subseteq Q$, is the final or accept states.

Explanation of the transition function, δ :

If, for any $a \in \Sigma$, $\delta(q, a, z) = \{(p_1, \beta_1), (p_2, \beta_2), \dots, (p_k, \beta_k)\}$. This means intuitively that whenever the PDA is in state q reading input symbol a and z on top of the stack, it can nondeterministically for any i , $1 \leq i \leq k$

- go to state p_i
- pop z off the stack
- push β_i onto the stack (where $\beta_i \in \Gamma^*$) (The usual convention is that if $\beta_i = X_1 X_2 \dots X_n$, then X_1 will be at the top and X_n at the bottom.)
- move read head right one cell past the current symbol a .

Instantaneous Description (ID)

ID is an informal notation of how a PDA computes an input string and make a decision that string is accepted or rejected.

An instantaneous description is a triple (q, w, α) where:

q describes the current state.

w describes the remaining input.

α describes the stack contents, top at the left.

Turnstile Notation:

$\vdash$ sign describes the turnstile notation and represents one move.

$\vdash^*$ sign describes a sequence of moves.

For example,

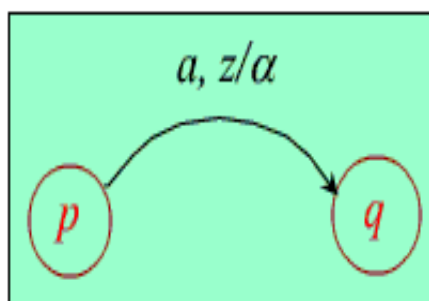
$(p, b, T) \vdash (q, w, \alpha)$

In the above example, while taking a transition from state p to q , the input symbol 'b' is consumed, and the top of the stack 'T' is represented by a new string α .

If $a = \epsilon$, then $\delta(q, \epsilon, z) = \{(p_1, \beta_1), (p_2, \beta_2), \dots, (p_k, \beta_k)\}$ means intuitively that whenever the PDA is in state q with z on the top of the stack regardless of the current input symbol, it can nondeterministically for any i , $1 \leq i \leq k$,

- go to state p_i
- pop Z off the stack
- push β_i onto the stack, and
- leave its read-only head where it is.

State transition diagram : A PDA can also be depicted by a state transition diagram. The labels on the arcs indicate both the input and the stack operation. The transition $\delta(p, a, z) = \{(q, \alpha)\}$ for $a \in \Sigma \cup \{\epsilon\}$, $p, q \in Q$, $z \in \Gamma$ and $\alpha \in \Gamma^*$ is depicted by



Language accepted by a PDA M

There are two alternative definitions of acceptance as given below.

1. Acceptance by final state :

Consider the PDA $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$. Informally, the PDA M is said to accept its input w by final state if it enters any final state in zero or more moves after reading its entire input, starting in the start configuration on input w .

Formally, we define $L(M)$, the language accepted by final state to be

$$\{ w \in \Sigma^* \mid (q_0, w, Z_0) \vdash_N^* (p, \epsilon, \beta) \text{ for some } p \in F \text{ and } \beta \in \Gamma^* \}$$

2. Acceptance by empty stack (or Null stack) : The PDA M accepts its input w by empty stack if starting in the start configuration on input w , it ever empties the stack w/o pushing anything back on after reading the entire input. Formally, we define $N(M)$, the language accepted by empty stack, to be

$$\{ w \in \Sigma^* \mid (q_0, w, z_0) \vdash_N^* (p, \epsilon, \epsilon) \text{ for some } p \in Q \}$$

Note that the set of final states, F is irrelevant in this case and we usually let the F to be the empty set i.e. $F = \emptyset$.

The following symbols are used while drawing the PDA

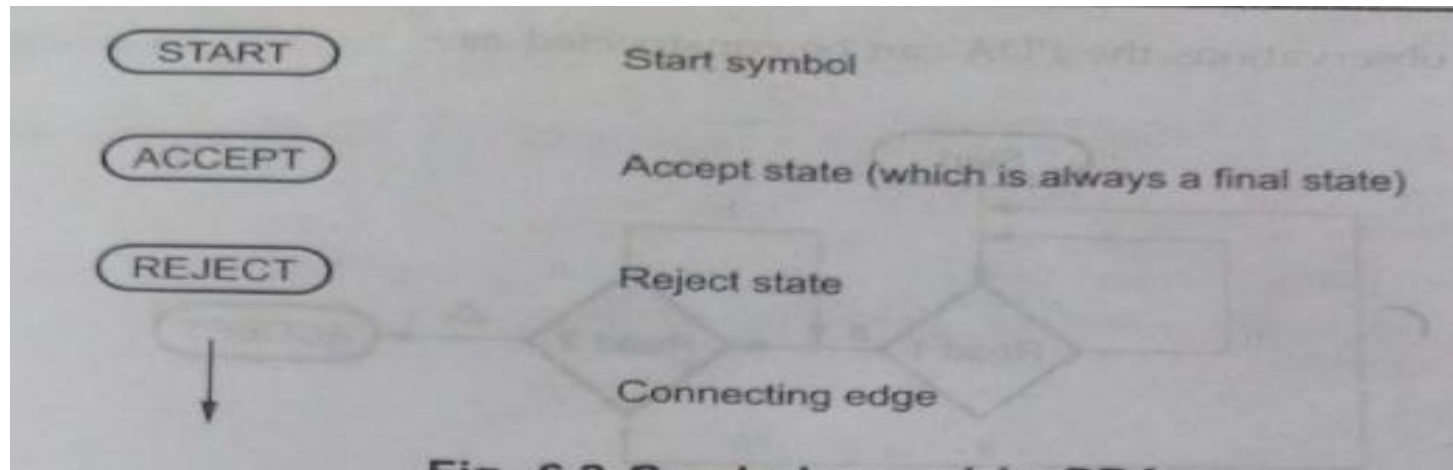


Fig. 6.2 Symbols used in PDA

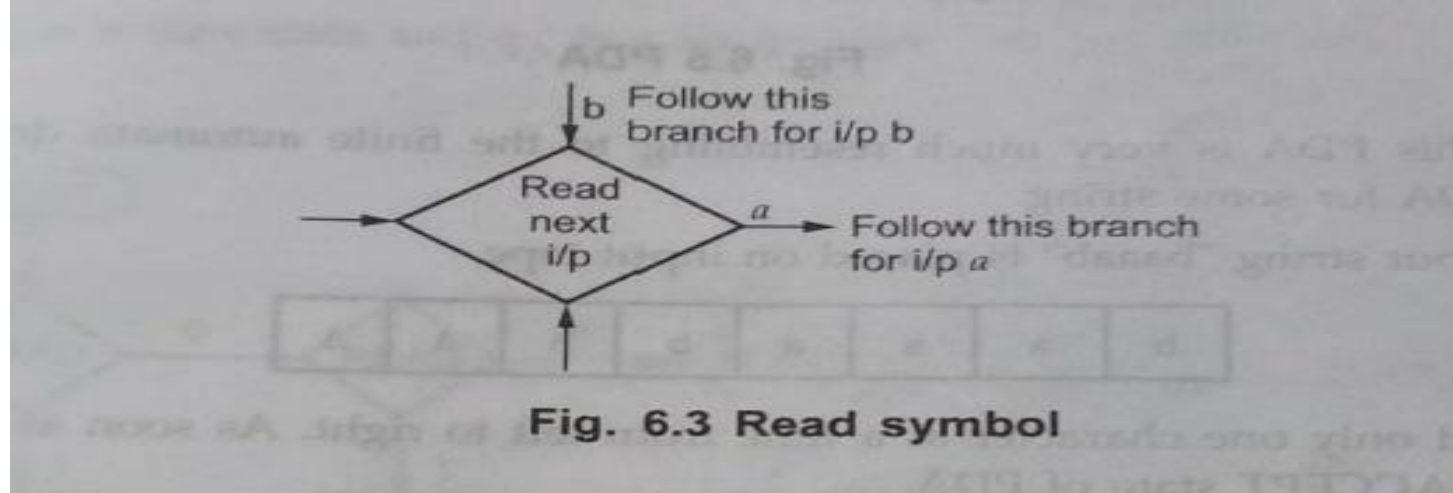


Fig. 6.3 Read symbol

➡ **Example 6.1 :** Design push down automata which accepts only odd number of a's over $\Sigma = \{a, b\}$.

Solution : This problem is similar to the one which we have solved for drawing FA. We will draw a finite automata for the same as -

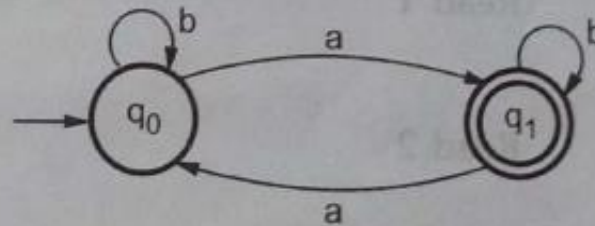


Fig. 6.4

We will first make some observations -

- If initially 'b' is read then the self loop is to be applied.
- If we read 'a' initially then the read state gets change [in FA state change from q_0 to q_1 occurs] and if again a is read then we return back to previous state.
- After a if we read ¹b then a self loop is applied.
- We basically read odd number of a's and any number of b's.

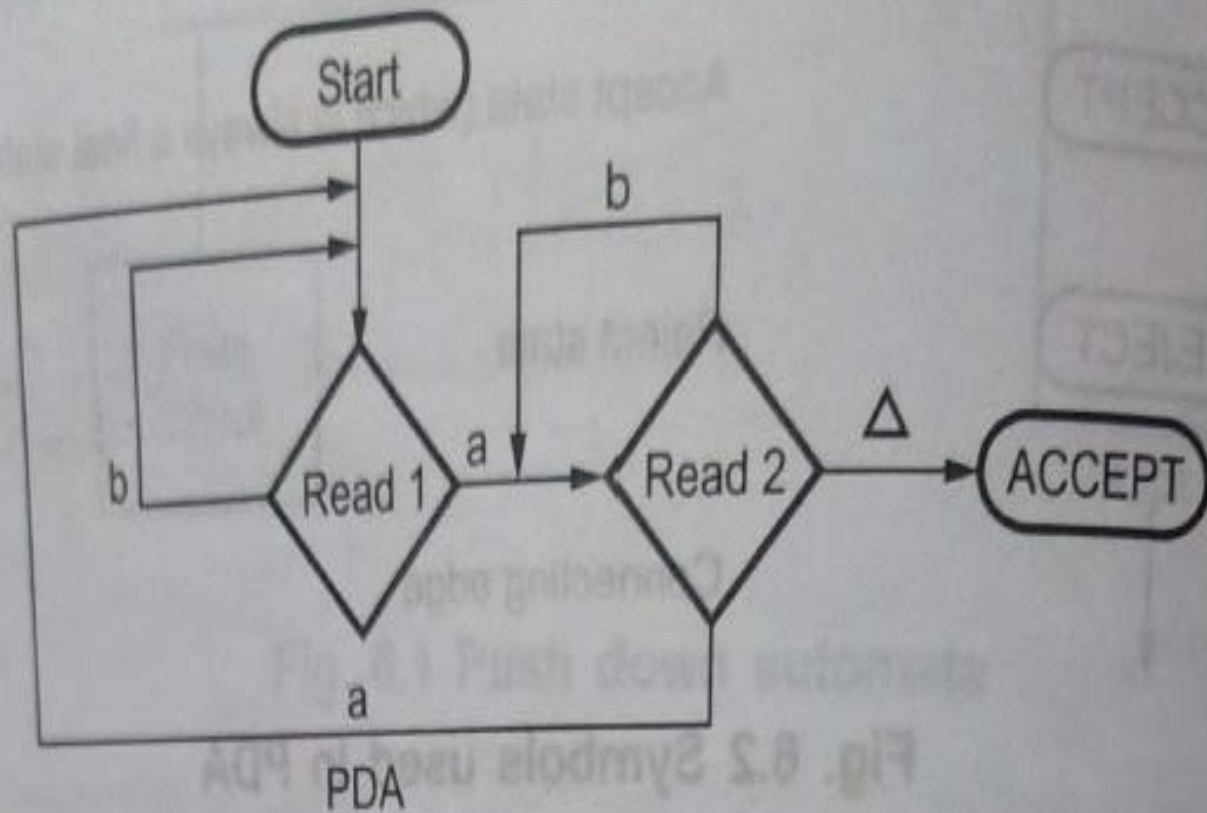


Fig. 6.5 PDA

Note that this PDA is very much resembling to the finite automata drawn. We simulate this PDA for some string.

Consider input string "baaab" is placed on input tape.

b	a	a	a	b	Δ	Δ	Δ
---	---	---	---	---	----------	----------	----------

We will read only one character at a time from left to right. As soon as we read Δ should reach to ACCEPT state of PDA.

b a a a b Δ

↑

Read 1

b a a a b Δ

↑

Read 2

b a a a b Δ

↑

Read 1

b a a a b Δ

↑

Read 2

b a a a b Δ

↑

Read 2

b a a a b Δ

↑

ACCEPT

This shows that only odd number of a's are allowed and there is no restriction on number of b's.

➔ **Example 6.3 :** Design a PDA for the language $L = \{a^n b^n\}$ where $n \geq 1$.

Solution : This is a typical example which can be solved by the PDA. Note that we cannot draw finite automata for this problem because, here the condition is that what many number of a 's are occurring those many b 's should be after a 's. That means if $n=5$ then the string will be $aaaaabbbbb$. If we try to design FA for this we are unable to keep track of how many a 's or b 's have occurred, since there is no memory in FA.

Let us draw PDA for the same using memory i.e. stack.

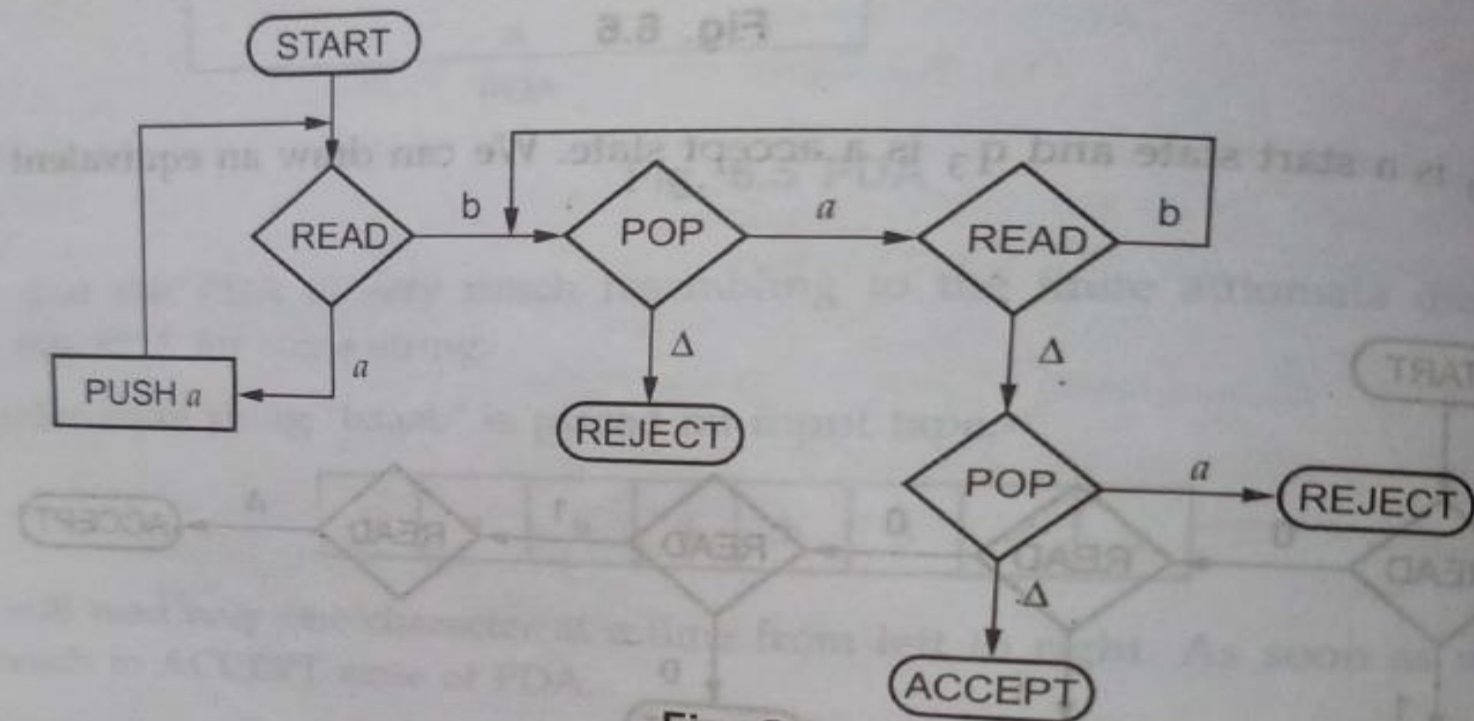
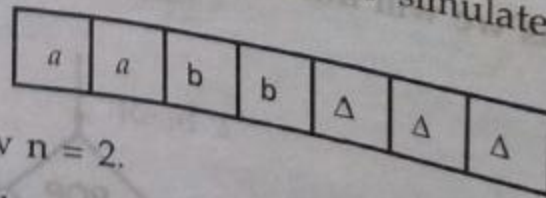


Fig. 6.8

Fig. 6.8

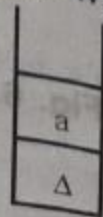
The simple logic we have applied here is that we are starting with start symbol. At the first read statement if we read a , we push that a onto the stack. This will be repeated by applying a self loop to this read statement. By this all the a 's will be pushed onto the stack. Now when we will read one b we will pop one a . Thus, this process will be repeated till the end of input. The end marker is Δ . After reading Δ from input tape, we pop the contents from the stack. If we pop a that means number of a 's are more than total number of b 's. So we go to a reject state. Let us simulate this machine for some valid string.



Input tape for $a^n b^n$, now $n = 2$.

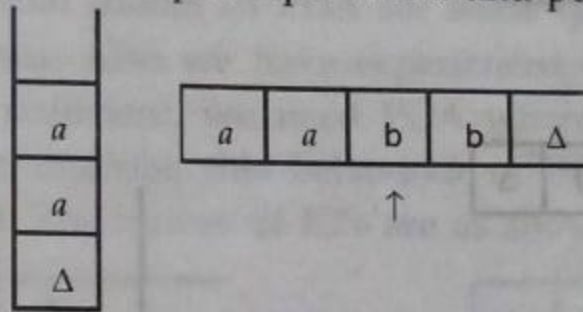
Step 1 : We will start with start symbol.

Step 2 : We will reach to a read state, we will read first a and push it onto the stack.

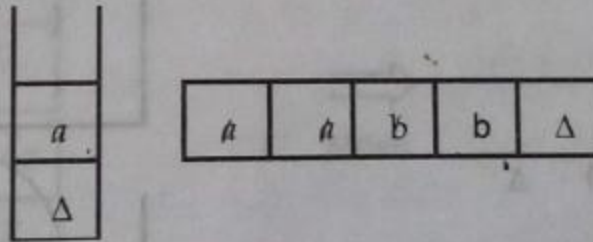


The stack initially, contains Δ that means it is empty.

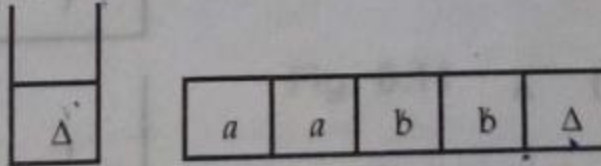
Step 3 : Read next symbol from input tape. It is a and push it onto the stack.



Step 4 : Read b from the tape and pop from the stack if the popped symbol is Δ it will go in reject states. That means number of a 's are lesser than b 's. So after reading first b , we pop one a .



After reading second b we pop one more a ,



Note that both tape and stack both are empty.

➡ **Example 6.6 :** Design a PDA for accepting a language $\{L = a^n b^n \mid n \geq 1\}$.

Solution : This is a language in which equal number of a's are followed by equal number of b's. The logic for this PDA can be applied as : first we will push all a's onto the stack . Then on reading every single b each a is popped from the stack. If we read all b and remove all a's and if we get stack empty then that string will be accepted. The instantaneous description can be given as -

$$\delta(q_0, a, Z_0) = (q_0, aZ_0)$$

$$\delta(q_0, a, a) = (q_0, aa)$$

$$\delta(q_0, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, Z_0) = (q_2, \epsilon)$$

Where q_0 is a start state and q_2 is accept state.

We will simulate this PDA for following string -

$$(q_0, aaabbbb, Z_0) \vdash (q_0, aabbbb, aZ_0)$$

$$\vdash (q_0, abbbb, aaZ_0)$$

$$\vdash (q_0, bbbb, aaaZ_0)$$

$$\vdash (q_1, bb, aaZ_0)$$

$$\vdash (q_1, b, aZ_0)$$

$$\vdash (q_1, \epsilon, Z_0)$$

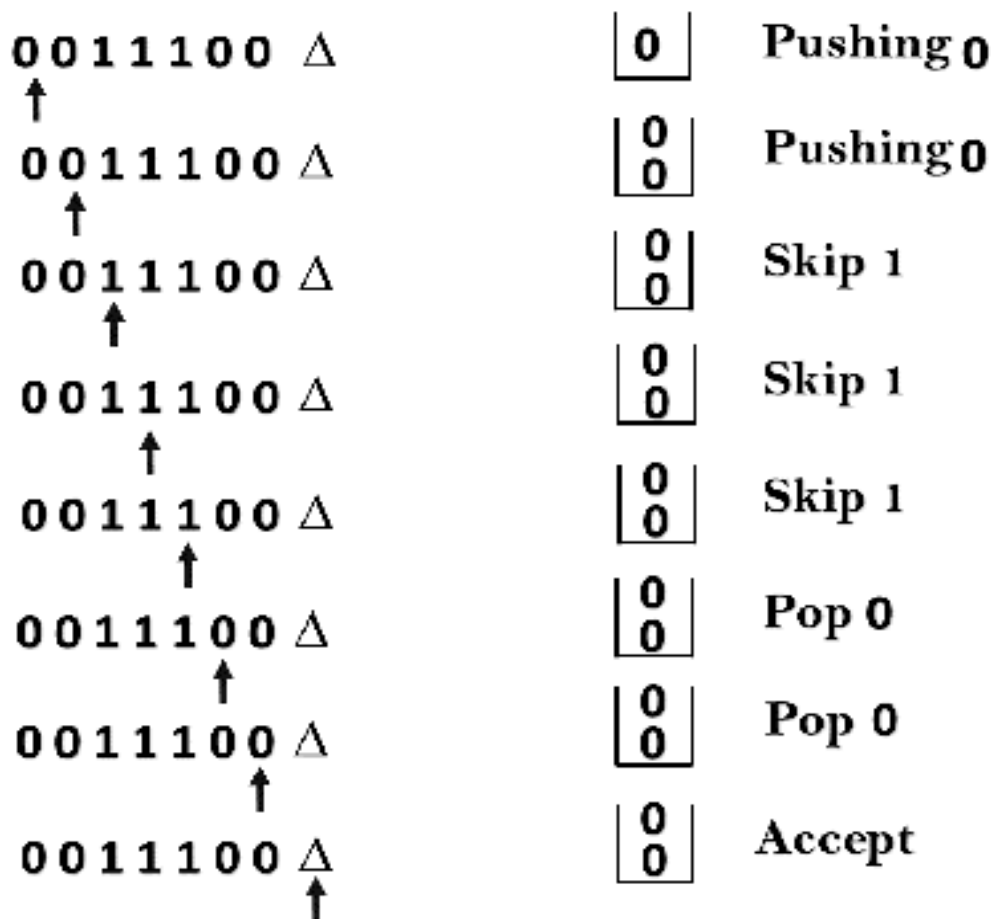
$$\vdash (q_2, \epsilon)$$

ACCEPT state.

Design A PDA for accepting a language $\{0^n 1^m 0^n \mid m, n \geq 1\}$.

Solution: In this PDA, n number of 0's are followed by any number of 1's followed n number of 0's. Hence the logic for design of such PDA will be as follows:

Push all 0's onto the stack on encountering first 0's. Then if we read 1, just do nothing. Then read 0, and on each read of 0, pop one 0 from the stack.



For instance:

This scenario can be written in the ID form as:

1. $\delta(q_0, 0, Z) = \delta(q_0, 0Z)$
2. $\delta(q_0, 0, 0) = \delta(q_0, 00)$
3. $\delta(q_0, 1, 0) = \delta(q_1, 0)$
4. $\delta(q_1, 1, 0) = \delta(q_1, 0)$
5. $\delta(q_1, 0, 0) = \delta(q_1, \epsilon)$
6. $\delta(q_1, \epsilon, Z) = \delta(q_2, Z)$ (ACCEPT state)

Now we will simulate this PDA for the input string "0011100".

1. $\delta(q_0, 0011100, Z) \vdash \delta(q_0, 011100, 0Z)$
2. $\vdash \delta(q_0, 11100, 00Z)$
3. $\vdash \delta(q_0, 1100, 00Z)$
4. $\vdash \delta(q_1, 100, 00Z)$
5. $\vdash \delta(q_1, 00, 00Z)$
6. $\vdash \delta(q_1, 0, 0Z)$
7. $\vdash \delta(q_1, \epsilon, Z)$
8. $\vdash \delta(q_2, Z)$
9. ACCEPT

CFG to PDA Conversion

The following steps are used to obtain PDA from CFG is:

Step 1: Convert the given productions of CFG into GNF.

Step 2: The PDA will only have one state $\{q\}$.

Step 3: The initial symbol of CFG will be the initial symbol in the PDA.

Step 4: For non-terminal symbol, add the following rule:

$\delta(q, \epsilon, A) = (q, \alpha)$, Where the production rule is $A \rightarrow \alpha$

Step 5: For each terminal symbols, add the following rule:

$\delta(q, a, a) = (q, \epsilon)$ **for** every terminal symbol

Ex: Construct PDA for the given CFG, and test whether 010^4 is acceptable by this PDA.

$S \rightarrow 0BB$

$B \rightarrow 0S \mid 1S \mid 0$

Solution:

The PDA can be given as:

$M = \{(q), (0, 1), (S, B, 0, 1), \delta, q, S, ?\}$

The production rule δ can be:

R1: $\delta(q, \epsilon, S) = \{(q, 0BB)\}$ **R2:** $\delta(q, \epsilon, B) = \{(q, 0S) \mid (q, 1S) \mid (q, 0)\}$ **R3:** $\delta(q, 0, 0) = \{(q, \epsilon)\}$

R4: $\delta(q, 1, 1) = \{(q, \epsilon)\}$

Testing 010^4 i.e. 010000 against PDA:

$\delta(q, 010000, S) \vdash \delta(q, 010000, 0BB)$

$\vdash \delta(q, 10000, BB)$	R1
$\vdash \delta(q, 10000, 1SB)$	R3
$\vdash \delta(q, 0000, SB)$	R2
$\vdash \delta(q, 0000, 0BBB)$	R1
$\vdash \delta(q, 000, BBB)$	R3
$\vdash \delta(q, 000, 0BB)$	R2
$\vdash \delta(q, 00, BB)$	R3
$\vdash \delta(q, 00, 0B)$	R2
$\vdash \delta(q, 0, B)$	R3
$\vdash \delta(q, 0, 0)$	R2
$\vdash \delta(q, \epsilon)$	R3
ACCEPT	

Thus 010^4 is accepted by the PDA.

Draw a PDA for the CFG given below:

- 1. $S \rightarrow aSb$
- 2. $S \rightarrow a \mid b \mid \epsilon$

Solution:

The PDA can be given as:

1. $P = \{(q), (a, b), (S, a, b, z_0), \delta, q, z_0, q\}$

The mapping function δ will be:

- R1:** $\delta(q, \epsilon, S) = \{(q, aSb)\}$
- R2:** $\delta(q, \epsilon, S) = \{(q, a) \mid (q, b) \mid (q, \epsilon)\}$
- R3:** $\delta(q, a, a) = \{(q, \epsilon)\}$
- R4:** $\delta(q, b, b) = \{(q, \epsilon)\}$
- R5:** $\delta(q, \epsilon, z_0) = \{(q, \epsilon)\}$

Simulation: Consider the string aaabb
 $\delta(q, \epsilon aaabb, S) \vdash \delta(q, aaabb, aSb)$

- $\vdash \delta(q, \epsilon aabb, Sb)$ R3
- $\vdash \delta(q, aabb, aSbb)$ R1
- $\vdash \delta(q, \epsilon abb, Sbb)$ R3
- $\vdash \delta(q, abb, abb)$ R2
- $\vdash \delta(q, bb, bb)$ R3
- $\vdash \delta(q, b, b)$ R4
- $\vdash \delta(q, \epsilon, z_0)$ R4
- $\vdash \delta(q, \epsilon)$ R5
- ACCEPT

Introduction to DCFL and DPDA

The Deterministic Push Down Automata (DPDA) is defined as M with 7-tuples:

$M=(Q, \Sigma, \Gamma, q_0, Z, F, \delta)$ where,

- **Q** : the finite set of states
- **Σ** : the input set
- **Γ** : a stack symbol which can be pushed and popped from the stack
- **q_0** : the initial state
- **Z** : a start symbol which is in Γ .
- **F** : a set of final states
- **δ** : mapping function which is used for moving from current state to next state.

The machine M is deterministic if $\delta(q, a, Z)$ has at most one element.

A language L is Deterministic Context Free Language(DCFL) if it is accepted by DPDA.

Turing Machine

- Turing machine was invented in 1936 by **Alan Turing**. It is an accepting device which accepts Recursive Enumerable Language generated by type 0 grammar.
- There are various features of the Turing machine:
 - It has an external memory which remembers arbitrary long sequence of input.
 - It has unlimited memory capability.
 - The model has a facility by which the input at left or right on the tape can be read easily.
 - The machine can produce a certain output based on its input. Sometimes it may be required that the same input has to be used to generate the output. So in this machine, the distinction between input and output has been removed. Thus a common set of alphabets can be used for the Turing machine.

Formal definition of Turing machine

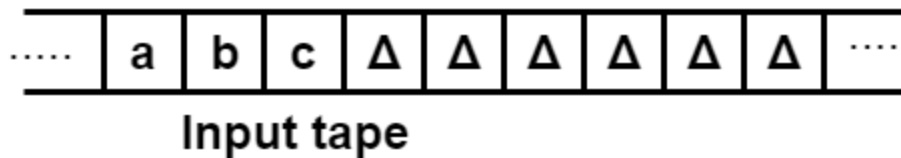
- A Turing machine can be defined as a collection of 7 components:
- **Q**: the finite set of states
 Σ : the finite set of input symbols
T: the tape symbol
q0: the initial state
F: a set of final states
B: a blank symbol used as a end marker for input
 δ : a transition or mapping function.
- The mapping function shows the mapping from states of finite automata and input symbol on the tape to the next states, external symbols and the direction for moving the tape head. This is known as a triple or a program for Turing machine.

$$(q_0, a) \rightarrow (q_1, A, R)$$

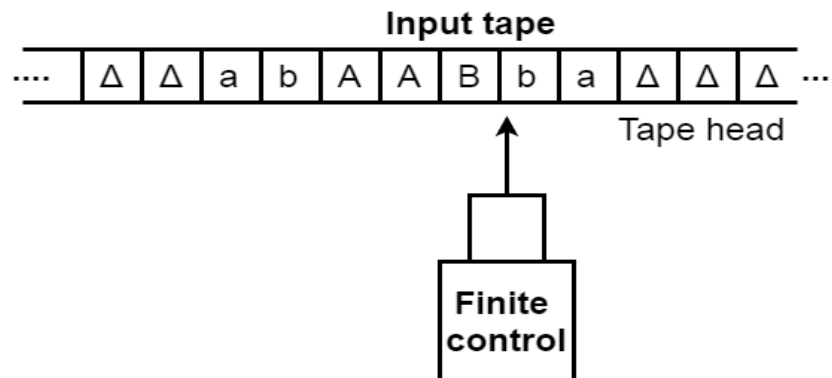
- That means in q_0 state, if we read symbol 'a' then it will go to state q_1 , replaced a by A and move ahead right(R stands for right).

Basic Model of Turing machine

- The Turing machine can be modeled with the help of the following representation.
- 1. The input tape is having an infinite number of cells, each cell containing one input symbol and thus the input string can be placed on tape. The empty tape is filled by blank characters.



- 2. The finite control and the tape head which is responsible for reading the current input symbol. The tape head can move to left to right.
- 3. A finite set of states through which machine has to undergo.
- 4. Finite set of symbols called external symbols which are used in building the logic of Turing machine.



Language accepted by Turing machine

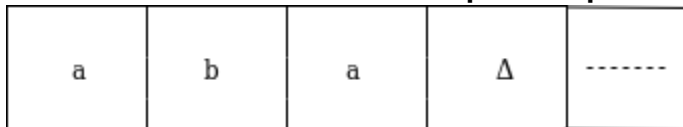
- The turing machine accepts all the language even though they are recursively enumerable.
- Recursive means repeating the same set of rules for any number of times and enumerable means a list of elements.
- The TM also accepts the computable functions, such as addition, multiplication, subtraction, division, power function, and many more.

Example:

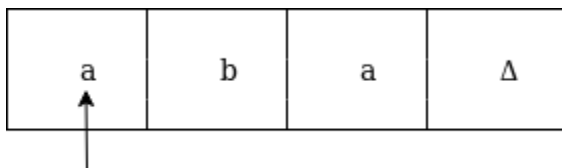
- Construct a turing machine which accepts the language of aba over $\Sigma = \{a, b\}$.

Solution:

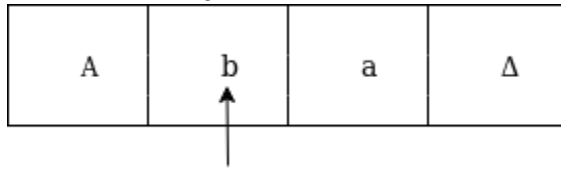
- We will assume that on input tape the string 'aba' is placed like this:



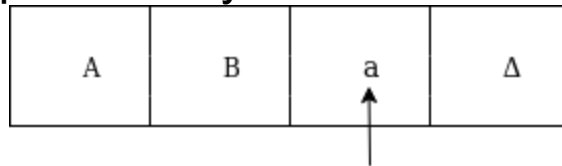
- The tape head will read out the sequence up to the Δ characters. If the tape head is readout 'aba' string then TM will halt after reading Δ .
- Now, we will see how this turing machine will work for aba. Initially, state is q_0 and head points to a as:



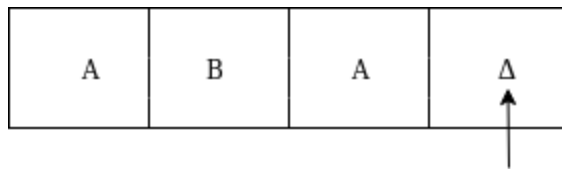
- The move will be $\delta(q_0, a) = \delta(q_1, A, R)$ which means it will go to state q_1 , replaced a by A and head will move to right as:



The move will be $\delta(q_1, b) = \delta(q_2, B, R)$ which means it will go to state q_2 , replaced b by B and head will move to right as:



The move will be $\delta(q_2, a) = \delta(q_3, A, R)$ which means it will go to state q_3 , replaced a by A and head will move to right as:

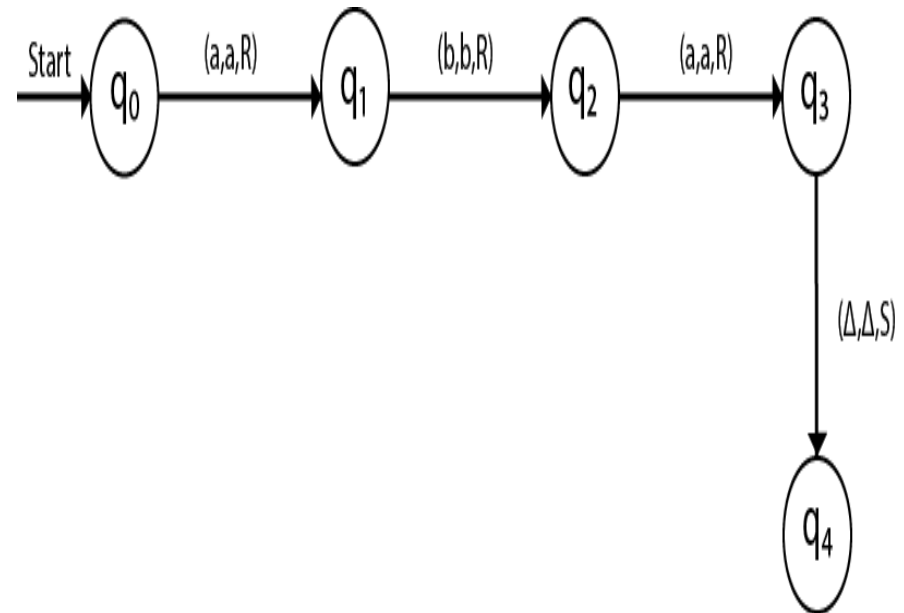


- The move $\delta(q_3, \Delta) = (q_4, \Delta, S)$ which means it will go to state q_4 which is the HALT state and HALT state is always an accept state for any TM.

Transition Table

States	a	b	Δ
q0	(q1, A, R)	–	–
q1	–	(q2, B, R)	–
q2	(q3, A, R)	–	–
q3	–	–	(q4, Δ , S)
q4	–	–	–

Transition Diagram

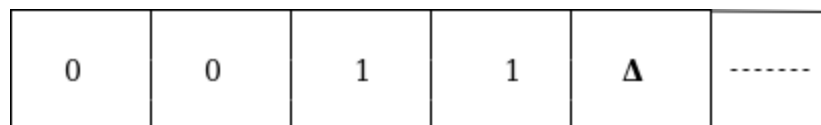


Example:

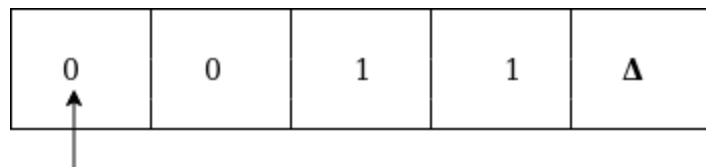
Construct TM for the language $L = \{0^n 1^n\}$ where $n \geq 1$.

Solution:

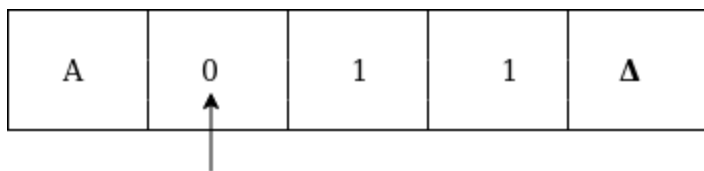
- We have already solved this problem by PDA. In PDA, we have a stack to remember the previous symbol. The main advantage of the Turing machine is we have a tape head which can be moved forward or backward, and the input tape can be scanned.
- The simple logic which we will apply is read out each '0' mark it by A and then move ahead along with the input tape and find out 1 convert it to B. Now, repeat this process for all a's and b's.
- Now we will see how this turing machine work for 0011.
- The simulation for 0011 can be shown as below:



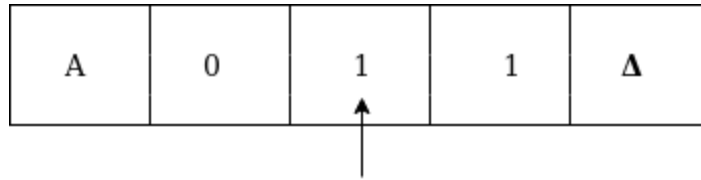
- Now, we will see how this turing machine will works for 0011. Initially, state is q_0 and head points to 0 as:



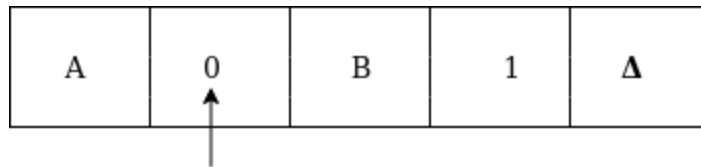
The move will be $\delta(q_0, 0) = \delta(q_1, A, R)$ which means it will go to state q_1 , replaced 0 by A and head will move to the right as:



- The move will be $\delta(q_1, 0) = \delta(q_1, 0, R)$ which means it will not change any symbol, remain in the same state and move to the right as:

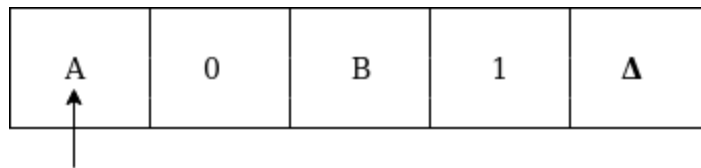


- The move will be $\delta(q_1, 1) = \delta(q_2, B, L)$ which means it will go to state q_2 , replaced 1 by B and head will move to left as:



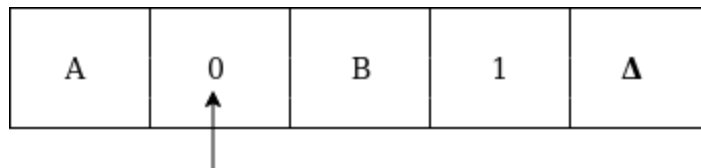
•

Now move will be $\delta(q_2, 0) = \delta(q_2, 0, L)$ which means it will not change any symbol, remain in the same state and move to left as:

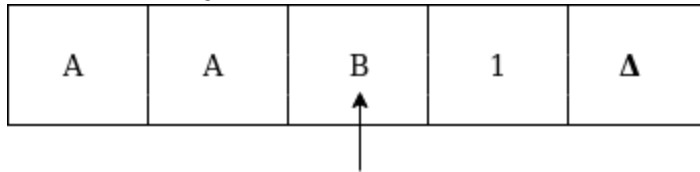


•

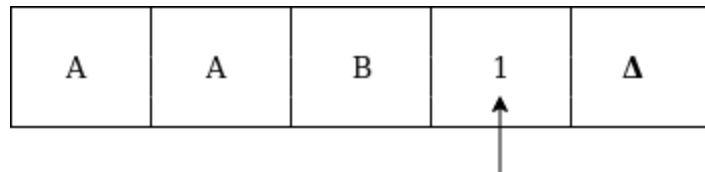
The move will be $\delta(q_2, A) = \delta(q_0, A, R)$, it means will go to state q_0 , replaced A by A and head will move to the right as:



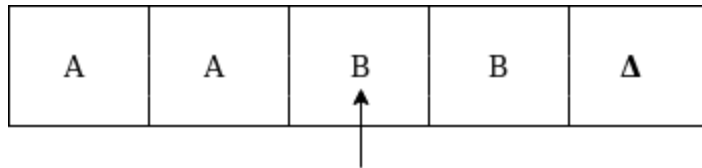
- The move will be $\delta(q_0, 0) = \delta(q_1, A, R)$ which means it will go to state q_1 , replaced 0 by A, and head will move to right as:



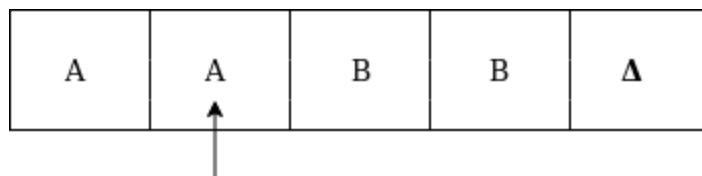
The move will be $\delta(q_1, B) = \delta(q_1, B, R)$ which means it will not change any symbol, remain in the same state and move to right as:



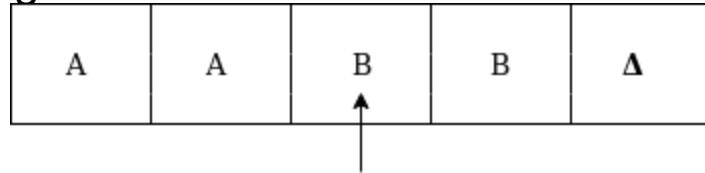
The move will be $\delta(q_1, 1) = \delta(q_2, B, L)$ which means it will go to state q_2 , replaced 1 by B and head will move to left as:



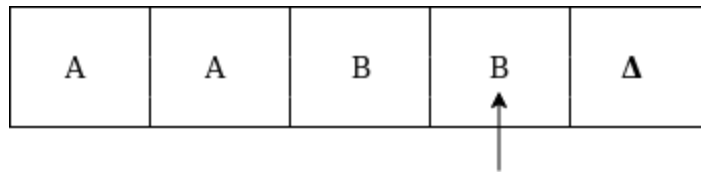
The move $\delta(q_2, B) = (q_2, B, L)$ which means it will not change any symbol, remain in the same state and move to left as:



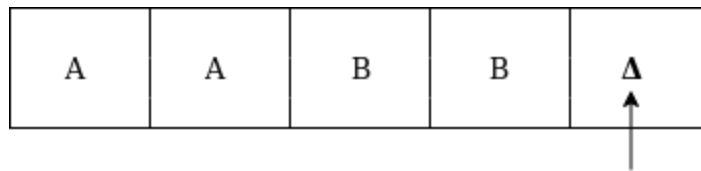
- Now immediately before B is A that means all the 0's are marked by A. So we will move right to ensure that no 1 is present. The move will be $\delta(q_2, A) = (q_0, A, R)$ which means it will go to state q_0 , will not change any symbol, and move to right as:



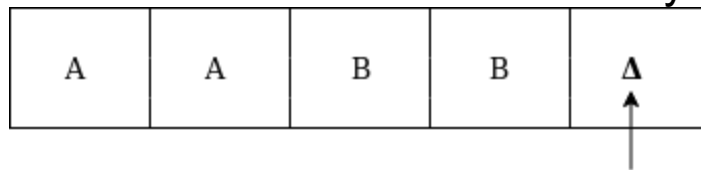
The move $\delta(q_0, B) = (q_3, B, R)$ which means it will go to state q_3 , will not change any symbol, and move to right as:



The move $\delta(q_3, B) = (q_3, B, R)$ which means it will not change any symbol, remain in the same state and move to right as:



- The move $\delta(q_3, \Delta) = (q_4, \Delta, R)$ which means it will go to state q_4 which is the HALT state and HALT state is always an accept state for any TM.



- Example 1:

Construct a TM for the language $L = \{0^n 1^n 2^n\}$ where $n \geq 1$

- Example 2:

Construct a TM machine for checking the palindrome of the string of even length.